Runtime Architecture Guide

This guide describes the current runtime architecture for the City Air Tracker application as implemented in the repository today.

Scheduling

The scheduling layer is intentionally thin. The shared orchestration runner lives in ‘services/pipeline/src/pipeline/orchestration/__init__.py‘. The active orchestration path is the Prefect runtime module in ‘services/pipeline/src/pipeline/prefect_runtime.py‘.

How scheduling integrates

- ‘services/pipeline/run_pipeline.py‘ is the top-level script entrypoint.
- ‘services/pipeline/src/pipeline/cli.py‘ parses command-line arguments and calls the shared ‘run_pipeline_job(...)‘.
- ‘services/pipeline/src/pipeline/prefect_runtime.py‘ exposes a Prefect flow wrapper around the shared ‘run_pipeline_job(...)‘.
- ‘services/pipeline/src/pipeline/orchestration/__init__.py‘ contains the actual pipeline flow.

This means manual CLI runs and Prefect-managed runs use the same orchestration path instead of duplicating pipeline logic.

Scheduling functions

‘pipeline.prefect_runtime.run_pipeline_flow(source="openweather", history_hours=None)‘

- Prefect flow wrapper around the shared orchestration runner.
- Delegates directly to ‘pipeline.orchestration.run_pipeline_job(...)‘.
- Provides the runtime entrypoint for ‘python -m pipeline.prefect_runtime‘ and Prefect-managed execution.

‘pipeline.orchestration.run_pipeline_job(source="openweather", history_hours=None)‘

- Main shared pipeline runner.
- Creates output directories, computes the runtime window, creates a pipeline-run tracking record, executes extract, transform, and load, then updates run status.
- Returns a ‘PipelineRunResult‘ with run metadata and publish targets.

‘pipeline.cli.main()‘

- CLI entrypoint for local or manual execution.
- Parses ‘--source‘, ‘--history-hours‘, and ‘--seed-cities‘.
- Calls the shared runner unless the command is explicitly seeding cities.

How scheduling can be tested

- ‘services/pipeline/tests/test_prefect_runtime.py‘ verifies the Prefect runtime wrapper delegates correctly.
- ‘services/pipeline/tests/test_orchestration_runner.py‘ verifies the shared orchestration flow.
- ‘services/pipeline/tests/test_run_pipeline_cities_file.py‘ verifies the CLI also routes into the shared runner.

Current limitation:

- The Prefect runtime entrypoint exists, but recurring configurable schedule registration is still a follow-up feature.

Extract Stage

The extract stage is coordinated by ‘run_extract_stage(...)‘ in ‘services/pipeline/src/pipeline/orchestration/__init__.py‘.

‘ensure_output_directories()‘

- Creates the raw and gold directories used by the pipeline runtime.
- Keeps the run from failing later due to missing output folders.

‘build_runtime_window(history_hours)‘

- Computes the UTC start and end timestamps for the extraction window.
- The current pipeline uses this to request OpenWeather historical pollution data for the configured time span.

‘run_extract_stage(raw_dir, start, end, run_id, pipeline_run_id)‘

- Reads the active cities, geocodes each city, and fetches the historical air-pollution payload for each location.
- Returns the list of raw records plus the number of processed cities.

‘extract.cities.read_cities(path=None)‘

- Chooses the configured city source.
- Reads from PostgreSQL when ‘CITIES_SOURCE=postgres‘, or from CSV when ‘CITIES_SOURCE=file‘.

‘extract.cities.read_cities_from_db()‘

- Reads active cities from the ‘cities‘ table.
- Returns ordered ‘CitySpec‘ records used by orchestration.

‘extract.cities.read_cities_file(path)‘

- Validates and reads the CSV city file.
- Converts rows into normalized ‘CitySpec‘ values.

‘extract.cities.seed_cities_from_file(path)‘

- Loads city definitions from CSV into PostgreSQL.
- Inserts new cities, skips duplicates, and reports counts in ‘CitySeedResult‘.

‘extract.geocoding.geocode_city(raw_dir, city, country_code, state=None)‘

- Resolves a city to latitude and longitude.
- Reuses cached coordinates from PostgreSQL when available, or calls the OpenWeather geocoding API and then upserts the cache.

‘extract.openweather_air_pollution.fetch_air_pollution_history(...)‘

- Retrieves historical air-pollution data for a city and time window.
- Reuses an existing raw response when the same city and window are already stored, otherwise calls the OpenWeather API and persists the raw payload in PostgreSQL.
- Returns a normalized ‘RawAirPollutionRecord‘.

Transform Stage

The transform stage is coordinated by ‘run_transform_stage(raw_records)‘ in ‘services/pipeline/src/pipeline/orchestration/__init__.py‘.

‘run_transform_stage(raw_records)‘

- Converts the extracted raw records into the gold analytical DataFrame.
- Delegates the real transformation work to ‘build_gold_from_raw_records(...)‘.

‘transform.openweather_air_pollution_transform.build_gold_from_raw_records(raw_records)‘

- Expands each raw payload into one row per observation timestamp.
- Builds normalized columns such as ‘aqi‘, pollutant measures, ‘geo_id‘, and location fields.
- Sorts rows, removes duplicate ‘(geo_id, ts)‘ observations, and computes a rolling ‘pm2_5_24h_avg‘.
- Calls the risk-scoring helpers before returning the final DataFrame.

‘transform.risk_scoring.add_aqi_category(df)‘

- Derives a human-readable AQI category from the pollution values.
- Adds a categorical label used by downstream consumers such as the dashboard.

‘transform.risk_scoring.add_risk_score(df)‘

- Computes a derived risk score from the transformed pollution data.
- Adds a higher-level metric that complements the raw pollutant values and AQI category.

Load Stage

The load stage is coordinated by ‘run_load_stage(gold_df, gold_dir, table_name="air_pollution_gold")‘ in ‘services/pipeline/src/pipeline/orchestration/__init__.py‘.

‘run_load_stage(gold_df, gold_dir, table_name="air_pollution_gold")‘

- Sends the transformed gold dataset to the configured output targets.
- Delegates persistence work to ‘publish_outputs(...)‘.

‘load.storage.publish_outputs(gold_df, gold_dir, table_name)‘

- Central load function for all publish targets.
- Writes the gold dataset to PostgreSQL when ‘USE_POSTGRES=1‘.
- Writes a local Parquet artifact when ‘WRITE_GOLD_PARQUET=1‘.
- Uploads a Parquet artifact to Azure Blob or Azurite when ‘WRITE_GOLD_AZURE_BLOB=1‘.
- Returns a ‘PublishResult‘ describing which targets were used.

‘load.storage._upsert_gold_rows(engine, gold_df, table_name)‘

- Prepares and upserts gold rows into PostgreSQL.
- Uses ‘INSERT ... ON CONFLICT (geo_id, ts) DO UPDATE‘ semantics so reruns update existing observations instead of replacing the full table.

‘load.storage._resolve_azure_blob_path(table_name)‘

- Builds the target blob path from ‘AZURE_BLOB_PATH‘.
- Supports templates such as ‘exports/{table_name}.parquet‘.

‘load.storage._upload_gold_to_azure_blob(gold_df, table_name)‘

- Creates the target blob container if needed.
- Converts the DataFrame to Parquet bytes in memory and uploads the artifact to Azure Blob Storage or Azurite.

Docker Compose Services

The local stack is defined in ‘docker-compose.yml‘.

‘postgres‘

- Local PostgreSQL database for the pipeline and dashboard.
- Stores cities, geocoding cache, raw API responses, pipeline runs, and gold analytical data.
- Includes a healthcheck so dependent services wait until the DB is ready.

‘migrate‘

- Runs Alembic migrations against PostgreSQL.
- Prepares the schema before pipeline execution.

‘pipeline‘

- Runs the ETL pipeline as a one-shot batch job.
- Waits for healthy PostgreSQL, successful migrations, and Azurite startup.

‘dashboard‘

- Runs the application dashboard.
- Reads from PostgreSQL and exposes the dashboard interface on port ‘8501‘.

‘adminer‘

- Web UI for inspecting PostgreSQL manually.
- Exposed on port ‘8080‘.

‘azurite‘

- Local Azure Blob emulator.
- Allows Azure-compatible Blob publishing to be developed and tested without a real Azure account.
- Exposed on port ‘10000‘.

‘azurestorageexplorer‘

- Browser-based storage explorer for Azurite.
- Used to inspect uploaded blobs locally at ‘http://localhost:8081‘.
- Configured with ‘AZURITE=true‘ and the shared Azurite connection string.

Architecture Review Notes

During this review, the architecture documentation was aligned with the current implementation in these areas:

- the system is now documented as DB-first rather than file-first
- the shared orchestration runner, manual CLI path, and Prefect runtime path are all represented
- load diagrams now include optional local Parquet and optional Azure Blob publishing
- the Docker Compose architecture now reflects ‘azurite‘ and ‘azurestorageexplorer‘

The remaining known gap is not documentation drift, but functionality: recurring Prefect schedule configuration is still a follow-up feature on top of the current shared runner and Prefect runtime entrypoint.